Research Gap Statement: Localization Failures and Wasted Cost in LLM-based Automated Program Repair

李睿祺 (L25020010) | Research Method, Lab 5 |
https://github.com/MuhammadRizkiEkaPratama/research-method-lab5-llm4apr.git

1. Background and State of the Art

Large language models (LLMs) are widely used in automated program repair (APR). Zhang et al. (2024) review 189 papers on LLM-based APR published between 2020 and 2025. A common benchmark is SWE-bench Lite, which contains 300 real GitHub issues drawn from 12 Python projects (Jimenez et al., 2024). Reported results have improved over time: SWE-agent with GPT-4 resolved 18.0% of issues, Agentless 1.5 with GPT-4o resolved 32.0%, and OpenHands with Claude 3.5 Sonnet resolved 41.7% (Yang et al., 2024; Xia et al., 2024; SWE-bench, 2024). On the full SWE-bench, the multi-agent framework MAGIS resolved 13.94% of issues. Its authors also report a relationship between successful repair and the model's ability to identify the relevant code (Tao et al., 2024). These results leave many issues unresolved, and unsuccessful runs still consume API resources.

2. The Specific Unresolved Gap

Some APR failures appear to occur before the system attempts a meaningful code change. In a pilot analysis, I compared the files modified by each leaderboard system's final patch with the files modified in the developer's reference fix, referred to here as the gold patch. Across 10 systems, 27%–58% of unresolved issues either produced no patch or produced a patch that did not touch any gold file (Figure 1). The cost-limit results from SWE-agent with GPT-4 show a related problem: 97 of 300 runs ended at the cost limit, and only 4 of those runs (4.1%) were resolved. Among runs that submitted normally, the resolution rate was 24.6%. These results suggest that some resources are spent on repair attempts before the system has located the files needed to solve the issue.

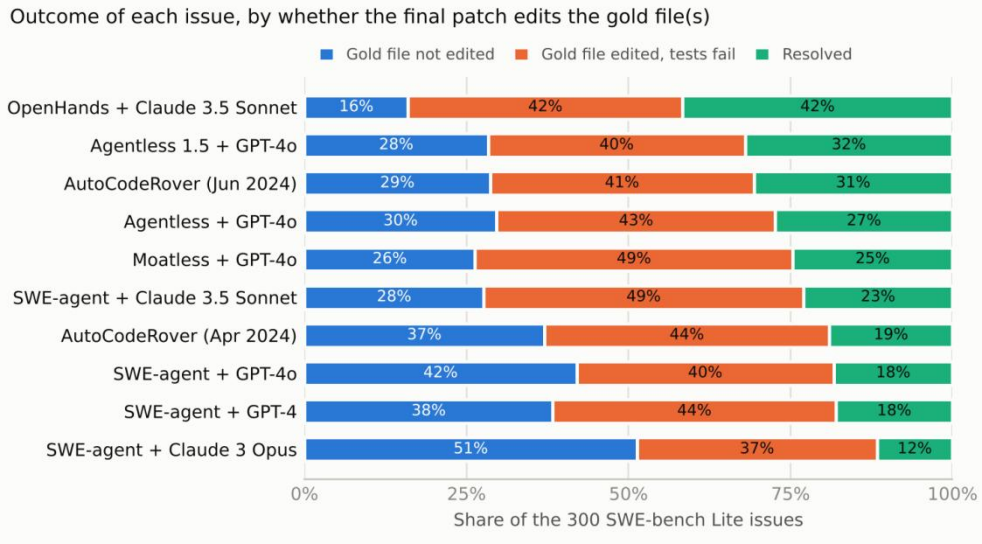


Figure 1. Outcomes for SWE-bench Lite issues across 10 leaderboard systems, grouped by whether the final patch modifies at least one gold file. Data: SWE-bench experiments repository; analysis by the author.

3. Why Existing Approaches Fail

The current pipeline does not appear to check whether localization has succeeded before moving on to repair. In Agentless 1.5 with GPT-4o, none of the 12,983 generated edits changed a file outside the locations supplied to the repair stage. If the relevant file is missed during localization, the repair stage therefore has no way to edit it. In the pilot data, the 60 issues (20%) whose gold file did not reach repair accounted for 23.9% of the 18.7 million tokens used, but only one of the 96 resolved issues (Figure 2). Their median token use was also higher than that of the remaining issues (64k compared with 55k; Mann–Whitney $p < 0.001$). Other studies have examined resource use or early stopping. Fan et al. (2025) report higher token use in unresolved runs, while EET (Guo et al., 2026) and Fail-Fast (Wang et al., 2026) use prior experience or signals from an agent's trajectory to stop runs early. The approach proposed here addresses a different point in the pipeline: checking localization before repair begins. This may be especially useful for pipeline-based systems and mid-sized models; Kim et al. (2026) report that stronger models can also fail after reaching the relevant code.

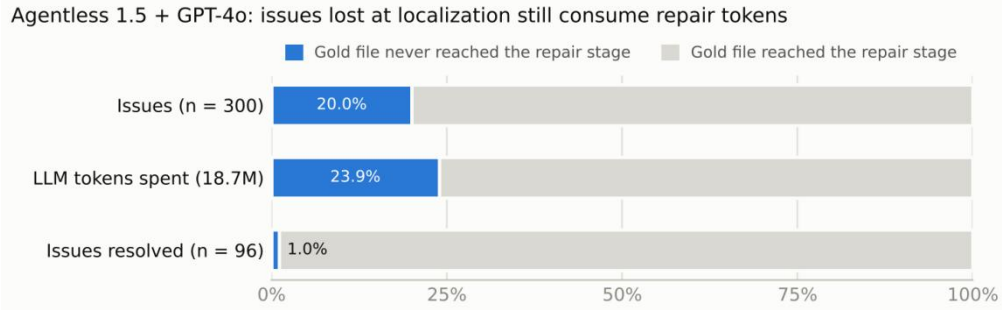


Figure 2. Agentless 1.5 with GPT-4o on SWE-bench Lite. The figure compares issues whose gold file did not reach repair with their share of total LLM tokens (prompt and completion; embeddings excluded) and resolved issues. Data: Agentless v1.5.0 release artifacts; analysis by the author.

4. Proposed Intervention and Expected Contribution

I propose a localization-confidence gate between the localization and repair stages. If the available evidence suggests that the system has probably missed the relevant files, the gate would either skip the repair attempt or request another set of locations. The pilot results offer one possible signal, although it is not strong enough on its own. When Agentless's four localization samples disagreed about which files to edit, the gold file was missed in 24.7% of cases; when the samples agreed, it was missed in 14.8% (Fisher exact $p = 0.042$). The study will test whether other signals, or combinations of signals, can make this decision more reliable. Since the Agentless outputs are public, the gate can first be evaluated by replaying the available artifacts offline, without making additional API calls. The evaluation will measure localization-related cost across systems, test a gate that does not use the gold patch at decision time, and compare the tokens saved with the number of otherwise solvable issues lost. Results will be compared with using no gate and with applying a fixed budget limit.

Research Questions

RQ1. How is reaching the gold file during localization associated with resolution outcomes and token use across LLM-based APR systems?

RQ2. How well can agreement between sampled localizations predict a localization miss when the gold patch is unavailable?

RQ3. At different gating thresholds, how many tokens can be saved and how many resolved issues may be lost, compared with no gate and a fixed budget cap?

RQ	Independent variable	Dependent variable	Data and analysis
RQ1	Gold file reaches repair (yes/no); system	Resolution rate; tokens per issue	SWE-bench Lite; 10 leaderboard systems and Agentless logs; Fisher exact and Mann–Whitney tests
RQ2	Agreement between localization samples	Localization miss (yes/no)	Agentless edit-location samples; precision, recall, ROC AUC
RQ3	Gate threshold	Tokens saved; resolved issues lost	Offline replay of Agentless artifacts; trade-off against a fixed budget cap

References

Fan, Z., Vasilevski, K., Lin, D., Chen, B., Chen, Y., Zhong, Z., Zhang, J. M., He, P., & Hassan, A. E. (2025). *SWE-Effi: Re-evaluating software AI agent system effectiveness under resource constraints*. arXiv:2509.09853.

Guo, Y., Xiao, Y., Zhang, J. M., Harman, M., Lou, Y., Liu, Y., & Chen, Z. (2026). *EET: Experience-driven early termination for cost-efficient software engineering agents*. arXiv:2601.05777.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). *SWE-bench: Can language models resolve real-world GitHub issues?* ICLR 2024. arXiv:2310.06770.

Kim, M., Wang, D., Cui, S., Farmahinifarahani, F., Zhuo, T. Y., Garg, S., Ray, B., Mukherjee, R., & Kumar, V. (2026). *Coherence collapse: Diagnosing why code agents fail after reaching the right code*. arXiv:2603.24631.

SWE-bench. (2024). *SWE-bench experiments: Predictions and results of leaderboard submissions*. https://github.com/SWE-bench/experiments

Tao, W., et al. (2024). *MAGIS: LLM-based multi-agent framework for GitHub issue resolution*. NeurIPS 2024. arXiv:2403.17927.

Wang, C., Lyu, Y., He, J., Yang, Z., Zhong, C., Harel, Y., & Lo, D. (2026). *Fail-fast, restart-smart: Early failure prediction and restart for SWE agentic tasks*. arXiv:2608.03222.

Xia, C. S., Deng, Y., Dunn, S., & Zhang, L. (2024). *Agentless: Demystifying LLM-based software engineering agents*. arXiv:2407.01489. Release artifacts v1.5.0: https://github.com/OpenAutoCoder/Agentless

Yang, J., Jimenez, C. E., et al. (2024). *SWE-agent: Agent-computer interfaces enable automated software engineering*. NeurIPS 2024. arXiv:2405.15793.

Zhang, Q., Fang, C., Xie, Y., Ma, Y., Sun, W., Yang, Y., & Chen, Z. (2024). *A systematic literature review on large language models for automated program repair*. arXiv:2405.01466.

Analysis code and data pipeline: https://github.com/MuhammadRizkiEkaPratama/research-method-lab5-llm4apr.git